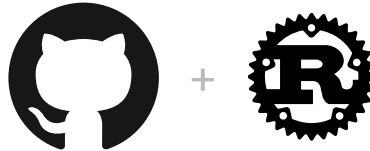


A Transpiler from C to Dafny

VectorPikachu

February 01, 2025



vectorpikachu/**Transpiler-from-C-to-Dafny**

A simple transpiler from C to Dafny.

Contents

1. Basic Usage	2
2. Translation Details	2
2.1. Arrays	2
3. Soundness Proof	2
Bibliography	3

1. Basic Usage

本项目使用 Rust 编写, 使用 cargo 进行构建. 首先, 确保你已经安装了 Rust 和 cargo.

```
1 cargo run -- -i <input_file> -o <output_file>
```

bash

2. Translation Details

我提供了一个 `traverse_tree()`, 它可以遍历语法树并打印出每个节点的类型. 我们可以通过这个函数来查看语法树的结构.

- `preproc_include` 类型是头文件
- `preproc_function_def` 类型是宏中的函数定义. e.g. `preproc_function_def`:
#define assume(e) if(!(e)) exit(-1);
- `preproc_def` 类型是宏定义. e.g. `preproc_def`: #define a (2)
- `function_definition` 类型是函数定义. e.g. `function_definition`:
int main()
{ return 0; }

我为每个表达式手动标记了类型, 以试图捕捉 C 语言中的强制类型转换, 并将其转换为 Dafny 中的 `as` 操作符. 我们可以在打印的时候加上这些操作符.

2.1. Arrays

关于数组: `seq<T>` 是 Dafny 中一个更加基本的类型, 而且虽然没有 `old` 操作符的支持, 但是我们可以通过下面的形式来模拟:

```
1  ghost var prevElements := a[..];
2  while // ...
3    invariant a[lo..hi] == prevElements[lo..hi]
4  {
5    // ...
6  }
7
8  // The above is equivalent to:
9  while // ...
10    invariant a[lo..hi] == old(a[lo..hi])
11  {
12    // ...
13  }
```

Dafny

但是 C 语言代码需要 `update` 一个数组, 这方面来看还是 `array<T>` 更合适, 所以我们选择使用 `array<T>`.

3. Soundness Proof

假设我们有这样几个转换:

- T_p : 从 C 的程序片段到 Dafny 的程序片段

- T_c : 从 C 的规约到 Dafny 的规约

对于任意 C 的霍尔三元组 $\langle c_1, p, c_2 \rangle$, 其对应的 Dafny 三元组为 $\langle T_c(c_1), T_p(p), T_c(c_2) \rangle$.

Soundness 定义为如果后者成立, 那么前者一定成立.

1. 现在假设 p 中出现了 `int` 类型, 并且出现了 `int` 的溢出操作, 那么 p 后面进行什么操作都是可以的, 因为这是一个 UB, 所以 c_2 无论如何都将成立, 所以我们可以直接把 C 语言中的 `int` 建模为 Dafny 中的 `int` 类型.
2. 如果 p 中出现了 `unsigned char` 和 `unsigned int`, 他们的溢出操作是有定义的回绕, 所以必须是 `bv` 类型.

Bibliography

- [1] N. Mohammed, A. Lal, A. Rastogi, R. Sharma, and S. Roy, “LLM Assistance for Memory Safety,” [Online]. Available: <https://api.semanticscholar.org/CorpusID:272603208>
- [2] N. Mohammed, A. Lal, A. Rastogi, S. Roy, and R. Sharma, “Enabling Memory Safety of C Programs using LLMs.” [Online]. Available: <https://arxiv.org/abs/2404.01096>
- [3] C. Sun, Y. Sheng, O. Padon, and C. Barrett, “Clover: Closed-Loop Verifiable Code Generation.” [Online]. Available: <https://arxiv.org/abs/2310.17807>
- [4] C. Loughridge *et al.*, “DafnyBench: A Benchmark for Formal Software Verification.” [Online]. Available: <https://arxiv.org/abs/2406.08467>
- [5] M. R. H. Misu, C. V. Lopes, I. Ma, and J. Noble, “Towards AI-Assisted Synthesis of Verified Dafny Methods,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 812–835, Jul. 2024, doi: [10.1145/3643763](https://doi.org/10.1145/3643763).
- [6] P. J. Chapman, C. Rubio-González, and A. V. Thakur, “Interleaving Static Analysis and LLM Prompting,” in *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, in SOAP 2024. Copenhagen, Denmark: Association for Computing Machinery, 2024, pp. 9–17. doi: [10.1145/3652588.3663317](https://doi.org/10.1145/3652588.3663317).
- [7] H. Wu, C. Barrett, and N. Narodytska, “Lemur: Integrating Large Language Models in Automated Program Verification.” [Online]. Available: <https://arxiv.org/abs/2310.04870>
- [8] N. Kosmatov, V. Prevosto, and J. Signoles, Eds., “Guide to Software Verification with Frama-C,” in *Computer Science Foundations and Applied Logic*. Springer Cham, 2024. doi: [10.1007/978-3-031-55608-1](https://doi.org/10.1007/978-3-031-55608-1).
- [9] DeepSeek-AI *et al.*, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [10] X. Shi, X. Xie, Y. Li, Y. Zhang, S. Chen, and X. Li, “Large-scale analysis of non-termination bugs in real-world OSS projects,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in ESEC/FSE 2022. Singapore, Singapore: Association for Computing Machinery, 2022, pp. 256–268. doi: [10.1145/3540250.3549129](https://doi.org/10.1145/3540250.3549129).

- [11] G. Poesia, C. Loughridge, and N. Amin, “dafny-annotator: AI-Assisted Verification of Dafny Programs.” [Online]. Available: <https://arxiv.org/abs/2411.15143>
- [12] E. Mugnier, E. A. Gonzalez, R. Jhala, N. Polikarpova, and Y. Zhou, “Laurel: Generating Dafny Assertions Using Large Language Models,” *CoRR*, 2024, doi: [10.48550/arxiv.2405.16792](https://arxiv.org/abs/2405.16792).
- [13] D. Brandfonbrener *et al.*, “VerMCTS: Synthesizing Multi-Step Programs using a Verifier, a Large Language Model, and Tree Search.” [Online]. Available: <https://arxiv.org/abs/2402.08147>
- [14] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software Testing With Large Language Models: Survey, Landscape, and Vision,” *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024, doi: [10.1109/TSE.2024.3368208](https://doi.org/10.1109/TSE.2024.3368208).
- [15] C. Cadar, D. Dunbar, and D. Engler, “KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs,” in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, in OSDI’08. San Diego, California: USENIX Association, 2008, pp. 209–224.
- [16] Y. C. Li, S. Zetsche, and S. Somayajula, “Dafny as Verification-Aware Intermediate Language for Code Generation.” [Online]. Available: <https://arxiv.org/abs/2501.06283>
- [17] S. Chakraborty *et al.*, “Towards Neural Synthesis for SMT-Assisted Proof-Oriented Programming.” [Online]. Available: <https://arxiv.org/abs/2405.01787>
- [18] S. K. Lahiri, “Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages.” [Online]. Available: <https://repositum.tuwien.at/handle/20.500.12708/200786>
- [19] A. F. Donaldson, D. Sheth, J.-B. Tristan, and A. Usher, “Randomised Testing of the Compiler for a Verification-Aware Programming Language,” in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024, pp. 407–418. doi: [10.1109/ICST60714.2024.00044](https://doi.org/10.1109/ICST60714.2024.00044).
- [20] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A Survey on Large Language Models for Code Generation.” [Online]. Available: <https://arxiv.org/abs/2406.00515>
- [21] W. Ahrendt, B. Beckert, R. Bubel, R. Hähnle, P. H. Schmitt, and M. Ulbrich, “Deductive Software Verification - The KeY Book,” in *Lecture Notes in Computer Science*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:34552701>
- [22] N. Piterman and S. A. Smolka, “Tools and Algorithms for the Construction and Analysis of Systems,” in *Lecture Notes in Computer Science*, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:37945079>
- [23] A. Kamath *et al.*, “Leveraging LLMs for Program Verification,” in *Formal Methods in Computer-Aided Design (FMCAD)*, Oct. 2024. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/finding-inductive-loop-invariants-using-large-language-models/>

- [24] C. Yang *et al.*, “AutoVerus: Automated Proof Generation for Rust Code.” [Online]. Available: <https://arxiv.org/abs/2409.13082>
- [25] T. Parr, *The Definitive ANTLR 4 Reference*. Manning Publications, 2013.
- [26] D. Beyer and M. Spiessl, “The Static Analyzer Frama-C in SV-COMP (Competition Contribution),” in *Tools and Algorithms for the Construction and Analysis of Systems*, D. Fisman and G. Rosu, Eds., Cham: Springer International Publishing, 2022, pp. 429–434.
- [27] W. Dietz, P. Li, J. Regehr, and V. Adve, “Understanding Integer Overflow in C/C++,” *ACM Trans. Softw. Eng. Methodol.*, vol. 25, no. 1, Dec. 2015, doi: [10.1145/2743019](https://doi.org/10.1145/2743019).
- [28] M. Surovič and F. Bertolaccini, “Rellic.” [Online]. Available: <https://github.com/lifting-bits/rellic>